

DBMS LAB – TASK 7 – CSE 3

SQL Views using the Taxation Database

1. Aim

To understand and apply SQL Views to simplify, secure, and reuse frequently required SQL queries.

2. Objective

By the end of this laboratory experiment, students should be able to:

- Understand the concept of SQL Views.
- Create views using SELECT queries.
- Retrieve data from views using SELECT statements.
- Create views using joins and aggregate functions.
- Understand the advantages and limitations of views.
- Solve real-world taxation queries using views for taxation analysis.

3. Concepts Covered

- SQL Views
- Creating Views
- Selecting from Views
- Views with WHERE Conditions
- Views with JOINS
- Views with Aggregate Functions
- Updating / Dropping Views
- View-based Analysis
- View Execution

4. Prerequisites

Students should already know:

- **SELECT, FROM, WHERE**

- Comparison Operators
- Logical Operators
- Aggregate Functions
- GROUP BY
- HAVING
- ORDER BY
- INNER JOIN, LEFT JOIN, RIGHT JOIN
- Table aliases

These concepts were covered in the previous laboratories. Task 7 introduced subqueries and nested queries. This task builds on those concepts by presenting reusable SQL Views.

5. Database to be Used

Continue using the existing database:

USE taxation_db;

Existing tables:

- Taxpayer
- Income_Record
- Income_Category
- Financial_Year

No new tables are required.

Insert if any additional data required as per Queries

6. Introduction to SQL Views

A view is a virtual table created from the result of a SELECT query.

A view stores the definition of a query and can be queried like a table. The underlying tables remain the source of the data.

General Syntax - CREATE VIEW

```
SELECT column_name
FROM table_name
WHERE column_name operator
      (SELECT column_name
       FROM table_name
       WHERE condition);
```

Example

Display taxpayers whose income is greater than the average income:

```
SELECT taxpayer_id, income_amount
FROM Income_Record
WHERE income_amount >
      (SELECT AVG(income_amount)
       FROM Income_Record);
```

Here:

- Inner query → calculates average income.
- Outer query → finds records greater than that average.

7. Creating and Using a View

A view can be created from a SELECT statement and then queried using SELECT.

It can generally be used with operators such as:

=
>

<
>=
<=
<>

Example

Display the income record having the highest income:

```
SELECT *  
FROM Income_Record  
WHERE income_amount =  
      (SELECT MAX(income_amount)  
        FROM Income_Record);
```

8. Views with JOINS

A view can combine columns from multiple tables using JOINS.

Operators such as:

IN
NOT IN
ANY
ALL

can be used with multi-row subqueries.

Example – IN

Display taxpayers who have Business income:

```
SELECT taxpayer_id  
FROM Income_Record  
WHERE category_id IN  
      (SELECT category_id  
        FROM Income_Category)
```

```
WHERE category_name = 'Business');
```

9. Views with Aggregate Functions

Aggregate functions learned in Task 5 can be used inside subqueries.

Examples:

```
AVG()  
MAX()  
MIN()  
SUM()  
COUNT()
```

Example

Display income records greater than the average income:

```
SELECT *  
FROM Income_Record  
WHERE income_amount >  
      (SELECT AVG(income_amount)  
       FROM Income_Record);
```

This creates the following flow:

```
Income Records  
  ↓  
AVG(income_amount)  
  ↓  
Average Income  
  ↓  
Outer Query  
  ↓  
Records above Average
```

10. Views with WHERE Conditions

IN checks whether a value exists in the result returned by a subquery.

Syntax

```
SELECT column_name
FROM table_name
WHERE column_name IN
    (SELECT column_name
     FROM table_name
     WHERE condition);
```

Example Owners:

Display income records belonging to taxpayers who are Business

```
SELECT *
FROM Income_Record
WHERE taxpayer_id IN
    (SELECT taxpayer_id
     FROM Taxpayer
     WHERE occupation = 'Business Owner');
```

11. Querying a View

NOT IN returns records whose value does not belong to the result of the subquery.

Example

Display taxpayers who have no income records in the Investment category:

```
SELECT *
FROM Taxpayer
WHERE taxpayer_id NOT IN
```

```
(SELECT taxpayer_id  
FROM Income_Record  
WHERE category_id = 3);
```

12. Modifying and Dropping Views

ANY

The condition becomes true if it satisfies the comparison with **at least one value** returned by the subquery.

Example:

```
SELECT *  
FROM Income_Record  
WHERE income_amount > ANY  
      (SELECT income_amount  
      FROM Income_Record  
      WHERE category_id = 3);
```

ALL

The condition becomes true only if it satisfies the comparison with **all values** returned by the subquery.

Example:

```
SELECT *  
FROM Income_Record  
WHERE income_amount > ALL  
      (SELECT income_amount  
      FROM Income_Record  
      WHERE category_id = 3);
```

Note: Views with JOINS are introduced here as multi-row subquery operators. More complex set-comparison applications can be practiced in later experiments.

13. Laboratory Activities

Part A – Basic Views

Level 1 – Understanding

Task 1: Create a view displaying the income record having the highest income.

Hint: Use MAX() in the view definition.

Task 2: Create a view displaying the income record having the lowest income.

Hint: Use MIN() in the view definition.

Task 3: Create a view displaying income records greater than the average income.

Hint: Use AVG() in the view definition.

Task 4: Create a view containing income records equal to the highest recorded income.

Hint: Use MAX() with =.

Task 5: Create a view containing taxpayers whose occupation is Business Owner.

Hint: Use a WHERE condition.

Level 2 – Application

Task 1: Create a view displaying taxpayers who have at least one income record.

Task 2: Create a view displaying taxpayers who have income in the Business category.

Task 3: Create a view displaying income records belonging to the financial year 2025-2026.

Hint: Use a JOIN with Financial_Year.

Task 4: Create a view displaying income records whose amount is greater than the minimum Business income.

Task 5: Create a view displaying income records whose amount is less than the maximum Salary income.

Task 7: Create a view displaying taxpayers who have income records greater than the average income.

Task 7: Create a view displaying income categories that have at least one income record.

Task 8: Create a view displaying taxpayers who have no income records in the Investment category.

Level 3 – Medium to Advanced

Task 1: Create a view displaying the taxpayer having the highest recorded income.

Hint: Use MAX().

Task 2: Create a view displaying all income records having an amount greater than the average Business income.

Task 3: Create a view displaying taxpayers whose total income is greater than the average total income of all taxpayers.

Task 4: Create a view displaying income records having an amount greater than at least one Investment income record.

Task 5: Create a view displaying income records having an amount greater than every Investment income record.

Task 7: Create a view displaying the income category that contains the highest income record.

Task 7: Create a view displaying the financial year having the highest total income.

Hint: Use SUM() and GROUP BY.

Task 8: Create a view displaying taxpayers whose total recorded income is greater than the average total income of taxpayers.

14. Real-World Taxation Analysis using Views

Task 1 Identify the taxpayer who has the highest individual income.

Hint: Use `MAX()` with a subquery.

Task 2 Identify taxpayers whose income is above the overall average income.

Hint: Use `AVG()` with a subquery.

Task 3 Identify the income category containing the highest income record.

Hint: Use `MAX()` and a nested query.

Task 4 Identify taxpayers who have Business income but no Investment income.

Hint: Use Views with WHERE Conditions.

Task 5 Identify income records whose amount is greater than every Investment income record. **Hint:** Use `ALL`.

Task 6 Identify income records whose amount is greater than at least one Investment income record. **Hint:** Use `ANY`.

Task 7 Display the taxpayer(s) having the highest total income.

Hint: Use `SUM()`, `GROUP BY`, and a subquery.

Task 8 Generate a list of income records whose amount is above the average income of their corresponding category.

Hint: This introduces the idea of comparing a record with a calculated group value; **do not use a correlated subquery yet.**

15. Important Concept

Students should understand the progression:

Individual Records



Aggregate Functions



GROUP BY



HAVING



ORDER BY



Subquery



Single-Row Subquery



Multi-Row Subquery



IN / NOT IN



ANY / ALL



Nested Analysis

The key idea is:

Outer Query → asks the final question

Inner Query → provides the value(s) required to answer it

16. Task 7 Submission

Students should create:

StudentID-repo

- └─ Week-7-Lab

- └─ week7_views.sql